

Taller 2: Búsqueda local y juegos adversarios

Integrantes

Grupo #

Ariana Lucia Lozano Parga - 202410609 - al.lozanop1@uniandes.edu.co

Tatiana Martínez Lozano - 202321001 - t.martinezl@uniandes.edu.co

Camilo Romero Rodriguez - 202411065 - ca.romeror123@uniandes.edu.co

Julian Felipe Perez Osorio - 202626725 - j.perez54@uniandes.edu.co

Universidad de los Andes

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas y Computación

ISIS-1611 Inteligencia Artificial

18 de septiembre de 2026

Tabla de Contenido

Configuración inicial mediante ascenso de colina.....	3
Local peak.....	3
Plateau grid.....	4
Escape de óptimos locales mediante recocido simulado.....	5
Caso 1 - wide plateau.....	5
Caso 2 - metropolitan network.....	6
Optimización poblacional mediante algoritmos genéticos.....	6
Defensa adversaria mediante Minimax.....	8
Single terminal.....	8
Tiny Defense.....	10
Bottleneck.....	11
Deceptive distance.....	12
Poda alfa-beta y función de evaluación.....	13
Dual terminal.....	15
Medium defense.....	17
Uso IA.....	20

Link repositorio: https://github.com/CaRoRo123/Taller2_IA.git

Configuración inicial mediante ascenso de colina

Elemento	Descripción
Representación Solucion	Tupla binaria en la que un 1 representa módulo de estabilización y un 0 indica que no lo tiene, tal que $R = \{X_1, X_2, \dots, X_i\}$, donde $X = \{1, 0\}$.
Restricción de validez	Contener exactamente k bits
Función objetivo	$puntaje = cobertura - redundancia - exposición$
Relación Vecindad	Se genera moviendo un módulo a un lugar donde X_i sea 0

Local peak

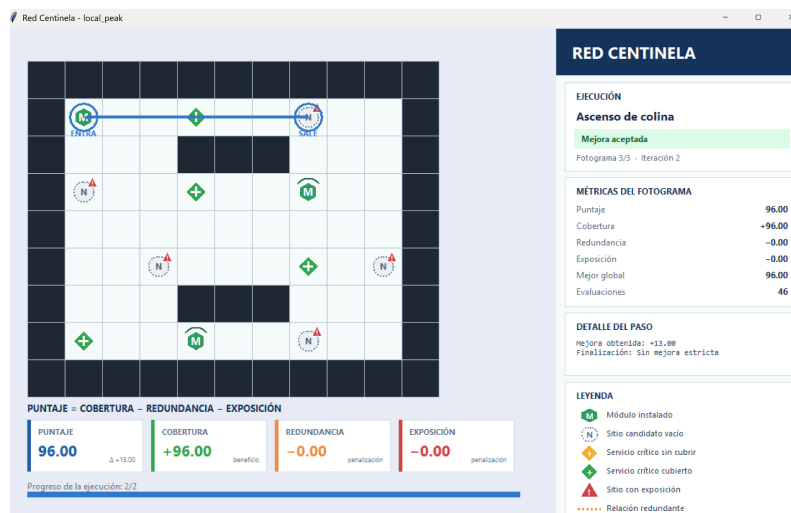


Imagen 1. Local peak 1

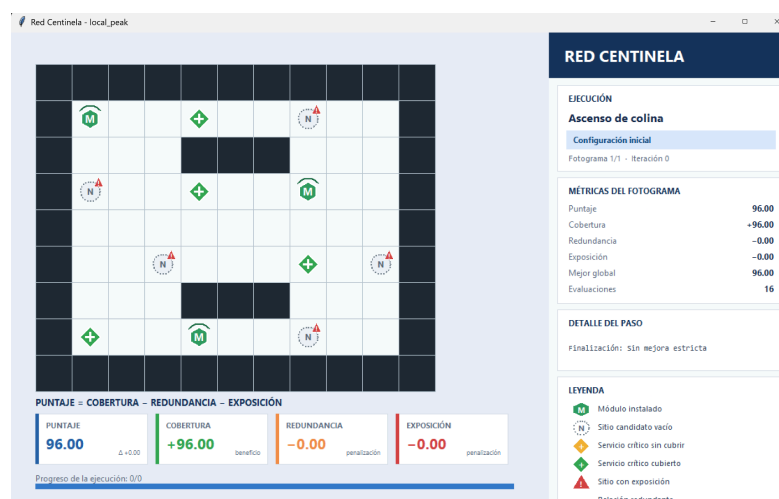


Imagen 2. Local peak 2

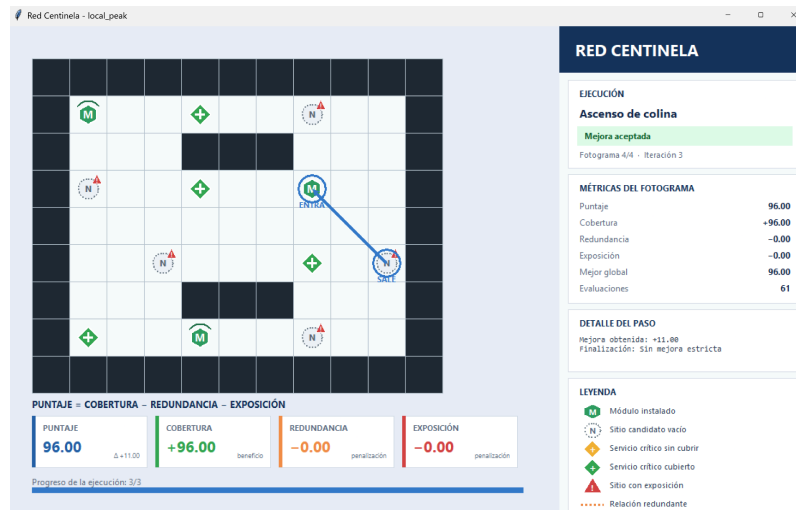


Imagen 3. Local peak 3

Plateau grid

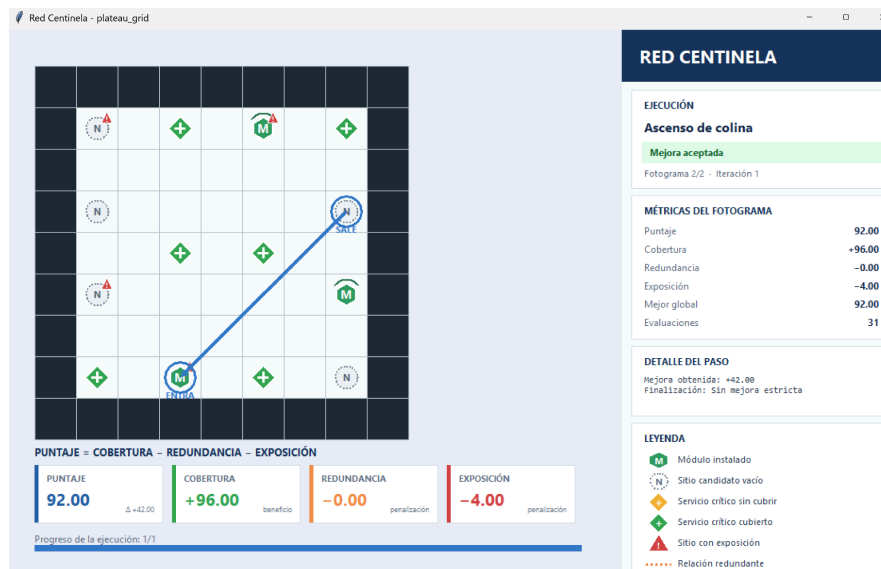


Imagen 4. Plateau grid 1

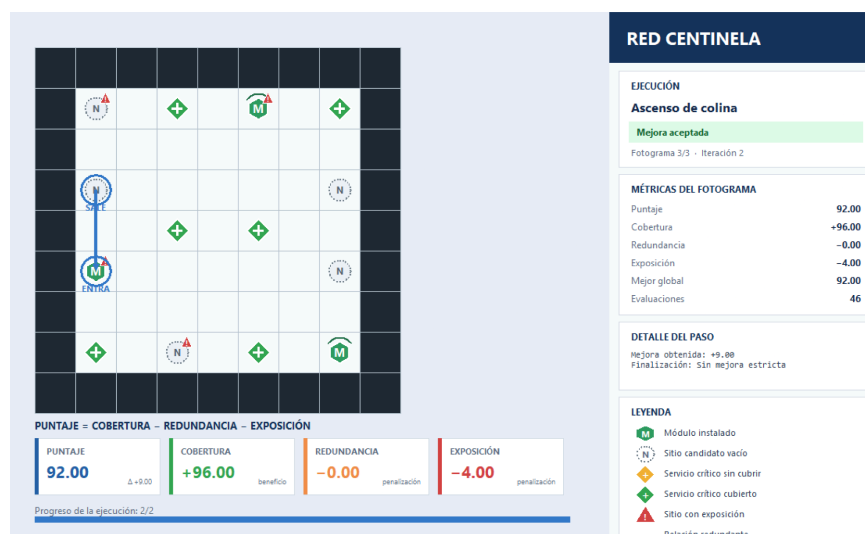


Imagen 5. Plateau grid 2

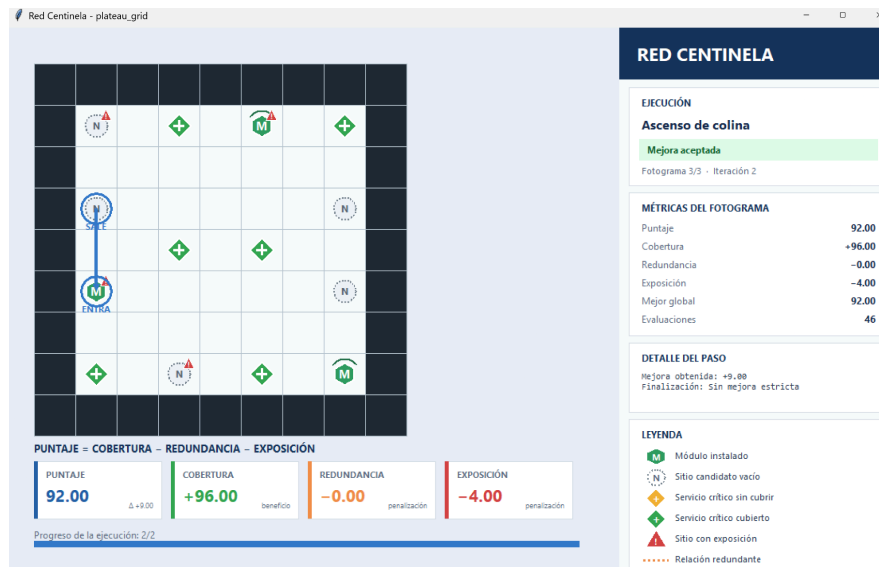


Imagen 6. Plateau grid 3

En las pruebas se aprecia cómo en ciertas ocasiones, no encuentra una mejora, o como la mejora no siempre es la misma mejora en las pruebas, esto se debe a que el algoritmo no permite salir de máximos locales.

Escape de óptimos locales mediante recocido simulado

Para este punto se creó el algoritmo con normalidad y se probó en modo texto con dos mapas, *wide plateau* y *metropolitan network*, y en cada uno de estos se probó con las tres versiones, para cambiar así su cooling rate y su temperatura. Los resultados fueron más constantes que en el uso de Hill climbing, pero de igual manera hubo cambios importantes con la solución que se encontró en todas las acciones.

Caso 1 - wide plateau

```
wide_plateau --seed 7 -n 1 -x 0.18 --iterations 250 --temperature 15 --cooling 0.95 -q
simulatedAnnealing | mejor=161.00 | promedio=161.00 | desviación=0.00 |
evaluaciones_promedio=251.0 | tiempo_promedio=0.030104s
```

```
wide_plateau --seed 10 -n 1 -x 0.18 --iterations 750 --temperature 20 --cooling 0.97 -q
simulatedAnnealing | mejor=161.00 | promedio=161.00 | desviación=0.00 |
evaluaciones_promedio=751.0 | tiempo_promedio=0.045863s
```

```
wide_plateau --seed 10 -n 1 -x 0.18 --iterations 2000 --temperature 30 --cooling 0.99 -q
simulatedAnnealing | mejor=161.00 | promedio=161.00 | desviación=0.00 |
evaluaciones_promedio=2001.0 | tiempo_promedio=0.189901s
```

Caso 2 - metropolitan network

```
metropolitan_network --seed 10 -n 1 -x 0.18 --iterations 250 --temperature 15 --cooling 0.95-q simulatedAnnealing | mejor=580.00 | promedio=580.00 | desviación=0.00 | evaluaciones_promedio=251.0 | tiempo_promedio=0.029127s
```

```
metropolitan_network --seed 10 -n 1 -x 0.18 --iterations 750 --temperature 20 --cooling 0.97-q simulatedAnnealing | mejor=571.00 | promedio=571.00 | desviación=0.00 | evaluaciones_promedio=751.0 | tiempo_promedio=0.077892s
```

```
metropolitan_network --seed 10 -n 1 -x 0.18 --iterations 2000 --temperature 30 --cooling 0.99-q simulatedAnnealing | mejor=580.00 | promedio=580.00 | desviación=0.00 | evaluaciones_promedio=2001.0 | tiempo_promedio=0.263386s
```

Optimización poblacional mediante algoritmos genéticos

En el algoritmo genético, la aleatoriedad inicial se ve reflejada en la población inicial que se escoge, así mismo a lo largo de las generaciones influyen en la escogencia de los padres que dan a lugar a la siguiente generación, así como el punto de corte que se va a usar para realizar el cruce de los padres. Debido a esta aleatoriedad, al cambiar la semilla se pueden llegar a obtener resultados distintos, especialmente en perfiles de búsqueda con baja profundidad.

Siguiendo la idea anterior, la profundidad indicada en los parámetros iniciales, la cual aumenta el tamaño de la población, el número de generaciones y, para el último perfil, la cantidad de individuos seleccionados por elitismo. Variar este perfil influye tanto en el tiempo de ejecución del algoritmo como de la calidad de la solución que retorna.

De acuerdo con la respuesta anterior, el tiempo de ejecución y la calidad de la solución están influenciadas por el perfil de búsqueda que se utilice. En este caso, se observa que en cuanto se usa un perfil de búsqueda más profundo, las soluciones retornadas suelen ser más cercanas al máximo, sin embargo también tienen un coste computacional más alto por lo que se demoran más tiempo en devolver la solución.

Para mostrar esto, se ejecutó el algoritmo en los siguientes layouts, realizando 5 ejecuciones por cada perfil de profundidad con tres semillas distintas:

```
mejor=351.00 | promedio=342.60 | desviación=10.29 | evaluaciones_promedio=1620.0 | tiempo_promedio=0.088843s
```

Imagen 7. Barrier_districts - perfil de búsqueda rápido - semilla 10

```
mejor=354.00 | promedio=348.60 | desviación=9.37 | evaluaciones_promedio=1620.0 | tiempo_promedio=0.088314s
```

Imagen 8. Barrier_districts - perfil de búsqueda rápido - semilla 7

mejor=354.00 | promedio=352.80 | desviación=1.47 | evaluaciones_promedio=1620.0 | tiempo_promedio=0.090801s

Imagen 9. Barrier_districts - perfil de búsqueda rápido - semilla 3

mejor=354.00 | promedio=352.80 | desviación=1.47 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.429726s

Imagen 10. Barrier_districts - perfil de búsqueda equilibrado - semilla 10

mejor=354.00 | promedio=352.20 | desviación=1.47 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.425263s

Imagen 11. Barrier_districts - perfil de búsqueda equilibrado - semilla 7

mejor=354.00 | promedio=352.80 | desviación=1.47 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.424592s

Imagen 12. Barrier_districts - perfil de búsqueda equilibrado - semilla 3

mejor=354.00 | promedio=352.80 | desviación=1.47 | evaluaciones_promedio=40080.0 | tiempo_promedio=1.927574s

Imagen 13. Barrier_districts - perfil de búsqueda profunda - semilla 10

mejor=354.00 | promedio=352.20 | desviación=1.47 | evaluaciones_promedio=40080.0 | tiempo_promedio=1.951847s

Imagen 14. Barrier_districts - perfil de búsqueda profunda - semilla 7

mejor=354.00 | promedio=352.80 | desviación=1.47 | evaluaciones_promedio=40080.0 | tiempo_promedio=1.954602s

Imagen 15. Barrier_districts - perfil de búsqueda profunda - semilla 3

mejor=580.00 | promedio=573.80 | desviación=3.66 | evaluaciones_promedio=1620.0 | tiempo_promedio=0.112350s

Imagen 16. Metropolitanian_network - perfil de búsqueda rápida - semilla 10

mejor=580.00 | promedio=574.40 | desviación=4.59 | evaluaciones_promedio=1620.0 | tiempo_promedio=0.112099s

Imagen 17. Metropolitanian_network - perfil de búsqueda rápida - semilla 7

mejor=580.00 | promedio=569.40 | desviación=9.00 | evaluaciones_promedio=1620.0 | tiempo_promedio=0.112863s

Imagen 18. Metropolitanian_network - perfil de búsqueda rápida - semilla 3

mejor=580.00 | promedio=580.00 | desviación=0.00 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.546285s

Imagen 19. Metropolitanian_network - perfil de búsqueda equilibrada- semilla 10

mejor=580.00 | promedio=580.00 | desviación=0.00 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.546255s

Imagen 20. Metropolitanian_network - perfil de búsqueda equilibrada- semilla 7

mejor=580.00 | promedio=576.00 | desviación=4.94 | evaluaciones_promedio=8040.0 | tiempo_promedio=0.543321s

Imagen 21. Metropolitanian_network - perfil de búsqueda equilibrada- semilla 3

mejor=580.00 | promedio=578.20 | desviación=3.60 | evaluaciones_promedio=40080.0 | tiempo_promedio=2.232365s

Imagen 22. Metropolitanian_network - perfil de búsqueda profunda - semilla 10

mejor=580.00 | promedio=580.00 | desviación=0.00 | evaluaciones_promedio=40080.0 | tiempo_promedio=2.231478s

Imagen 23. Metropolitanian_network - perfil de búsqueda profunda - semilla 7

mejor=580.00 | promedio=580.00 | desviación=0.00 | evaluaciones_promedio=40080.0 | tiempo_promedio=2.260042s

Imagen 24. Metropolitanian_network - perfil de búsqueda profunda - semilla 3

Defensa adversaria mediante Minimax

Componente	Definición
Estado inicial (S_0)	Posición inicial de defensor (MAX), del intruso (MIN) y conjunto de terminales pendientes por activar en el mapa.
Jugador en Turno ($Turno(s)$)	Función que indica a quién le toca mover: Agente 0 (Defensor) en niveles pares/MAX o Agente 1 (Intruso) en niveles impares/MIN.
Acciones ($Acciones(s)$)	Lista de movimientos legales devueltos por <code>state.get_legal_actions(agent_index)</code> (Arriba, Abajo, Izquierda, Derecha, Quedarse quieto).
Modo de transición/ Sucesor	Función <code>state.get_legal_actions(agent_index, action)</code> que genera el nuevo estado s' al aplicar la acción.
Prueba de estado terminal	Evaluación mediante <code>state.is_win()</code> (defensor activo todas las terminales) o <code>state.is_lose()</code> (intruso atrapó al defensor)
Utilidad	Puntaje asignado al estado: +1000 si gana el defensor, -1000 si es atrapado, o <code>evaluation_function(s)</code> si corta por profundidad.

Single terminal

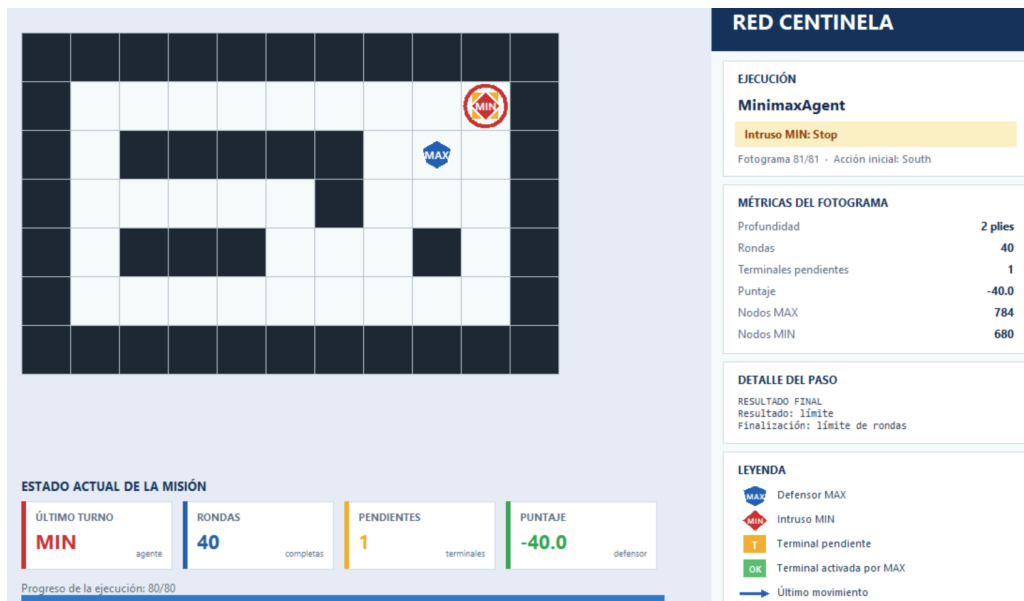


Imagen 25. Single_terminal profundidad 2

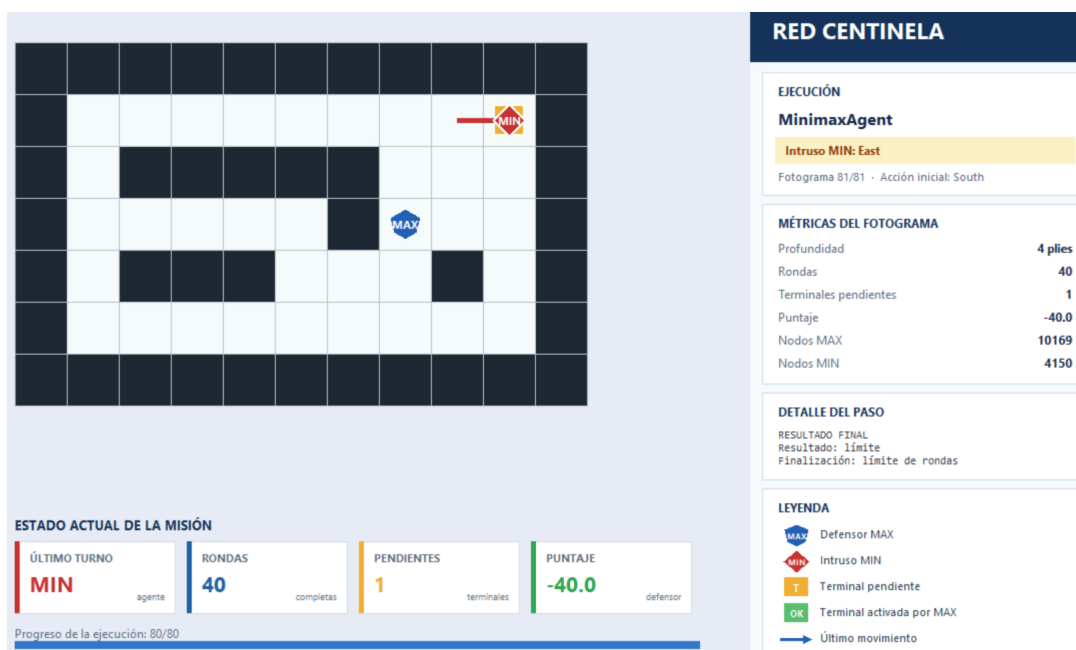


Imagen 26. Single_terminal profundidad 4

Tiny Defense

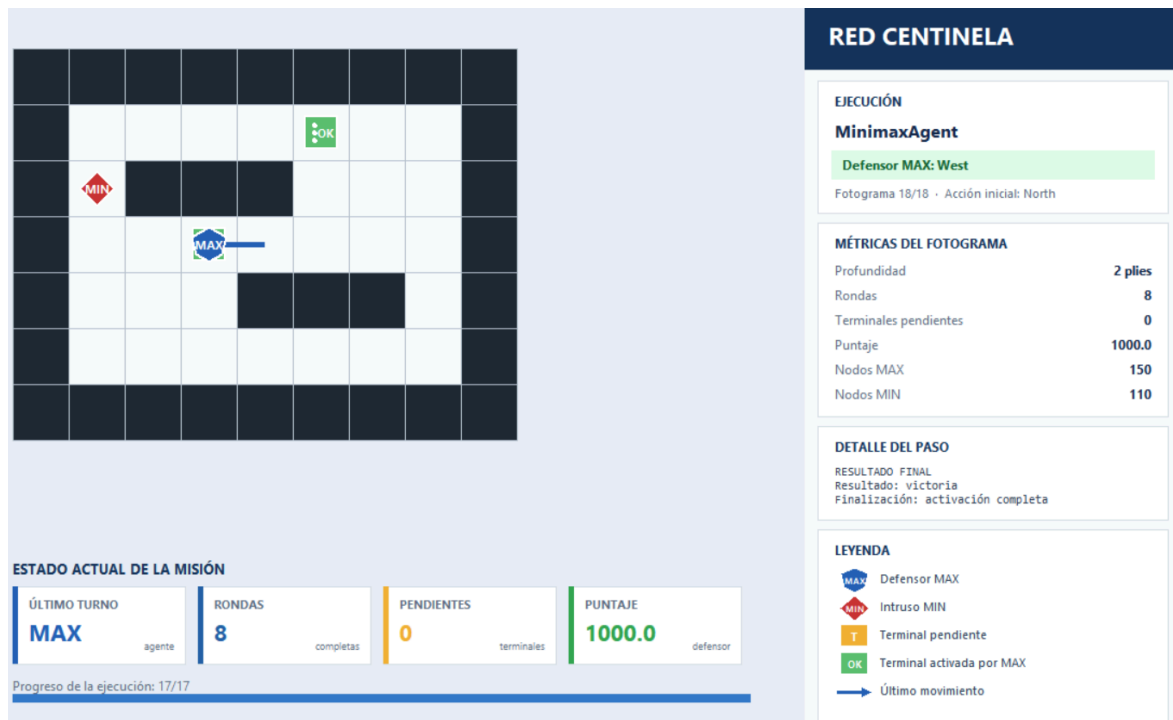


Imagen 27. Tiny defense profundidad 2

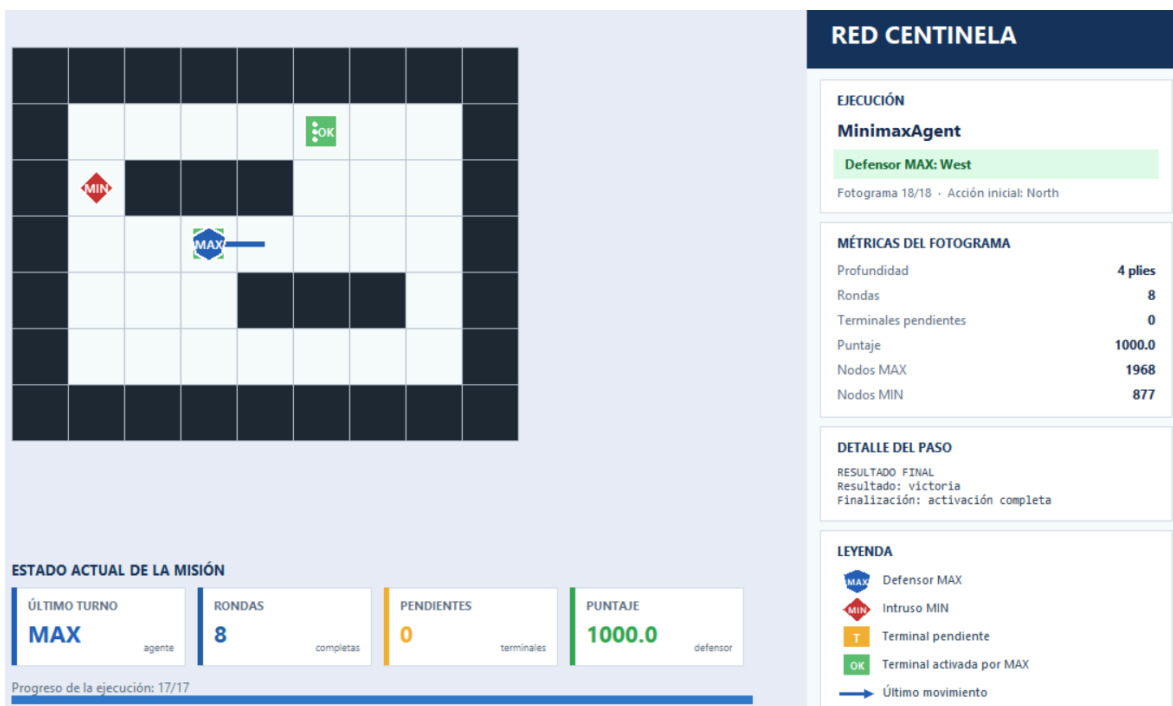


Imagen 28. Tiny defense profundidad 4

Bottleneck

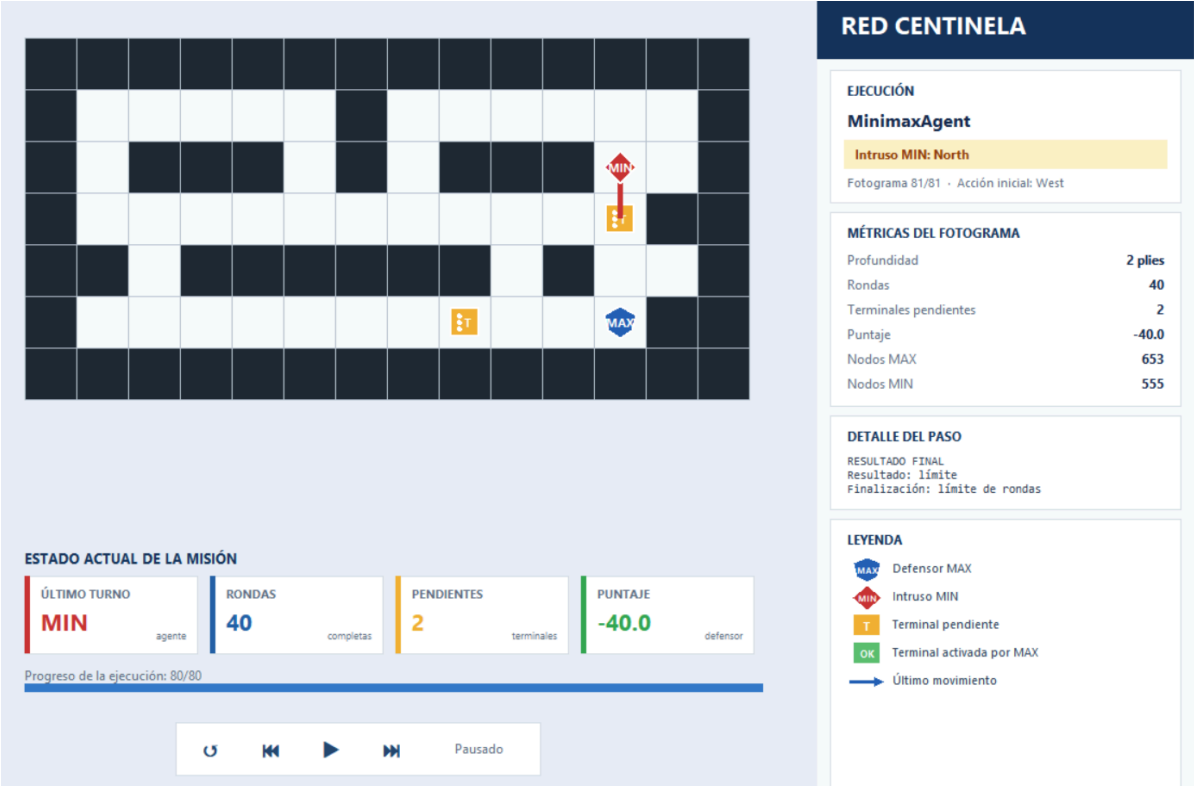


Imagen 29. Bottleneck profundidad 2

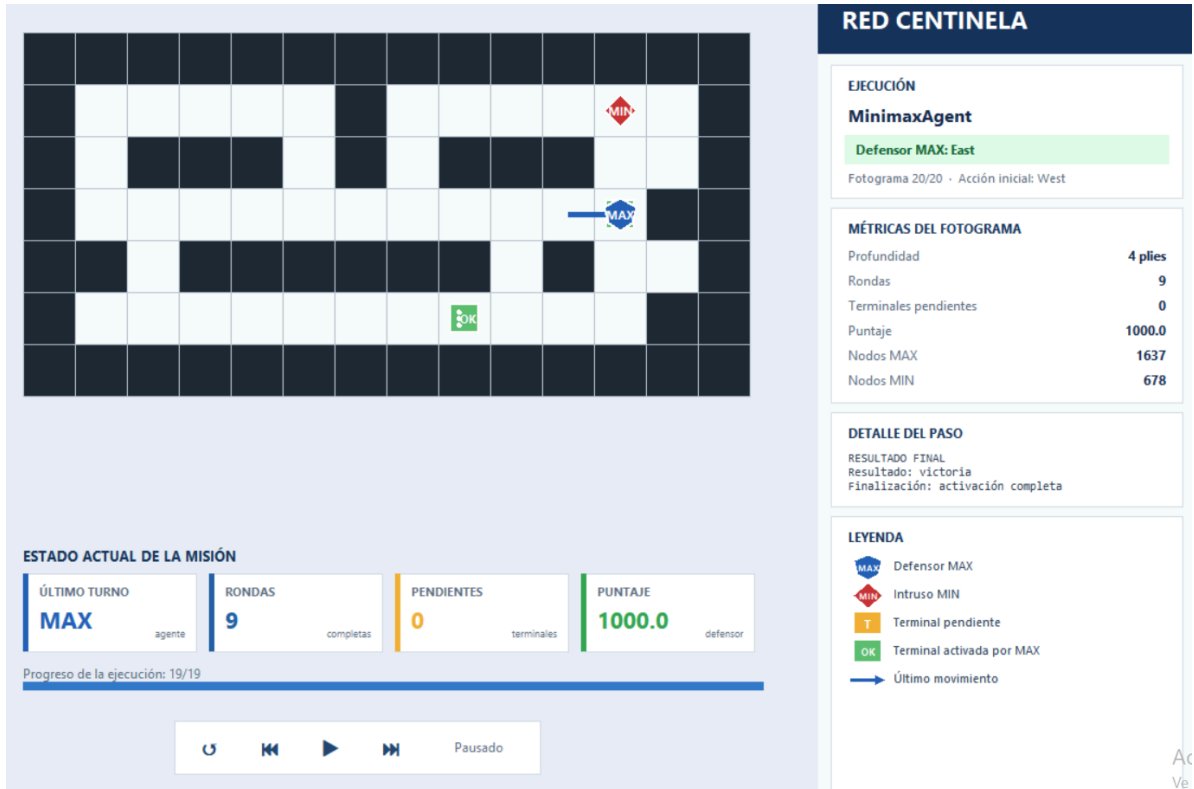


Imagen 30. Bottleneck profundidad 4

Deceptive distance

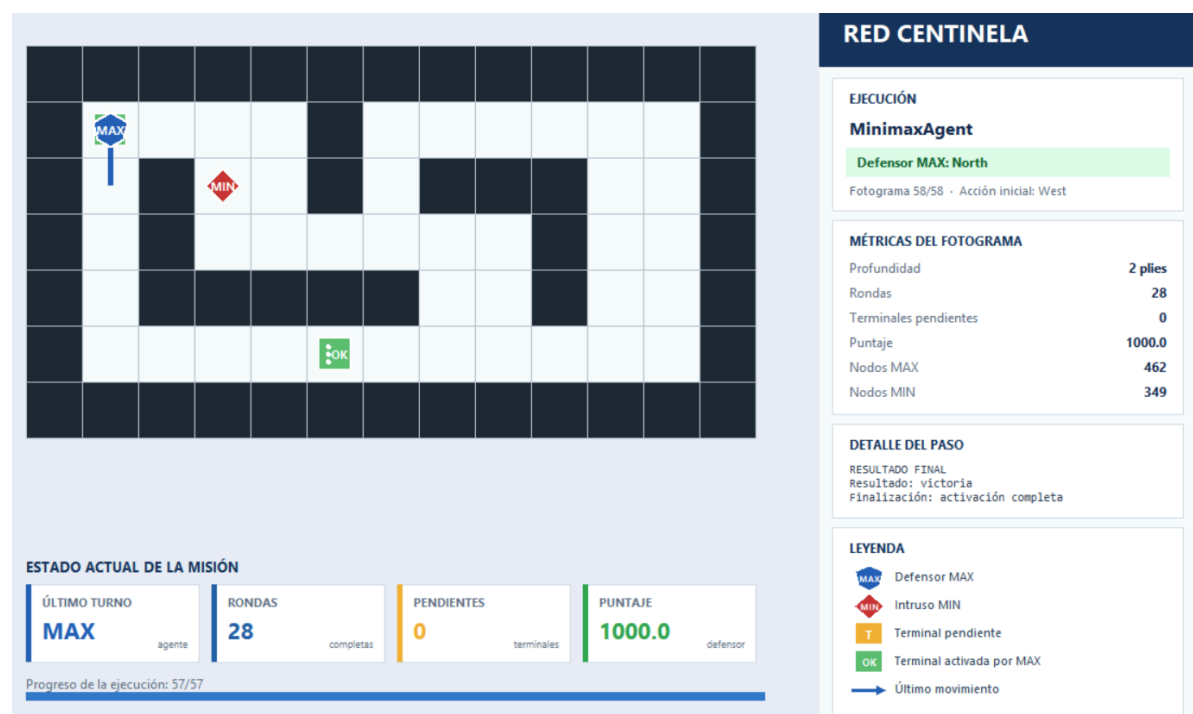


Imagen 31. Deceptive distance profundidad 2

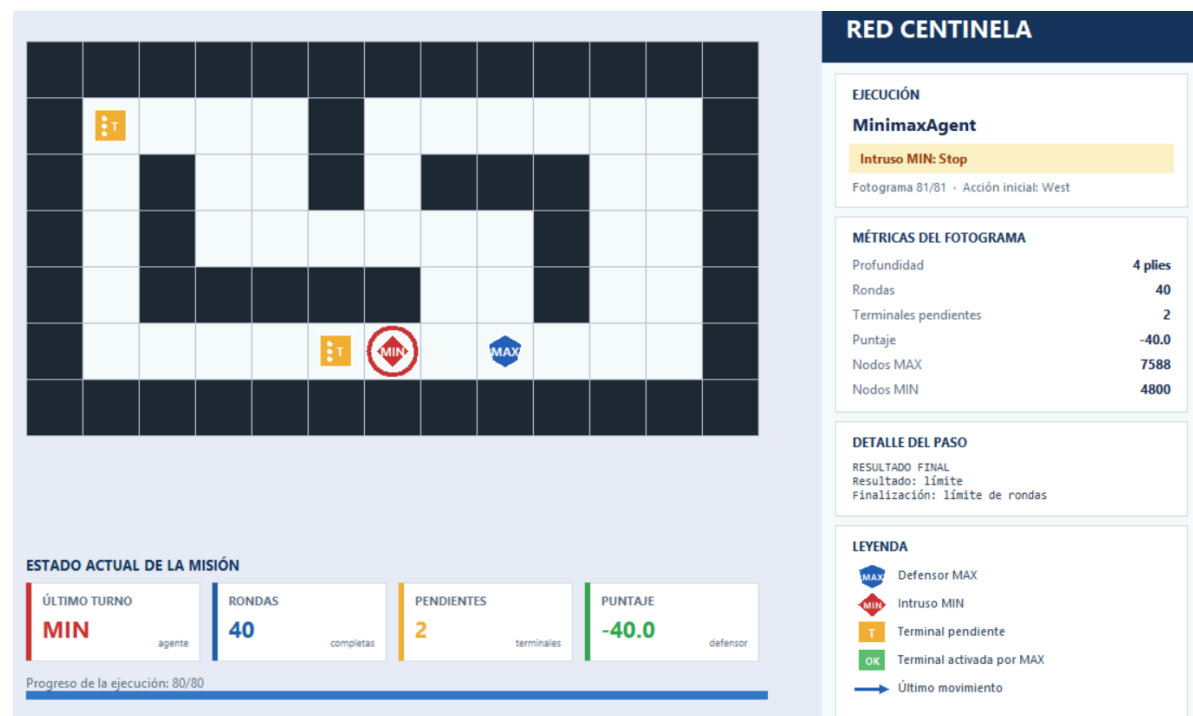


Imagen 32. Deceptive distance profundidad 4

Escenario	Profundidad	Acción inicial	Resultado Final	Nodos MAX explorados	Nodos MIN explorados
Single terminal	2	South	Límite de rondas (Puntaje: -40.0)	784	860
Single terminal	4	South	Límite de rondas (Puntaje: -40.0)	10169	4150
Tiny Defense	2	North	Victoria (Puntaje:+1000)	150	110
Tiny Defense	4	North	Victoria (Puntaje:+1000)	1968	877
Bottleneck	2	West	Límite de rondas (Puntaje: -40.0)	653	555
Bottleneck	4	West	Victoria (Puntaje:+1000)	1637	678
Deceptive distance	2	West	Victoria (Puntaje:+1000)	462	349
Deceptive distance	4	West	Límite de rondas (Puntaje: -40.0)	7588	4800

El comportamiento de Minimax se resume en tres factores clave:

- **Topología:** Mapas simples (tiny defense) permiten victorias directas con poco esfuerzo computacional (150 nodos). En mapas complejos (bottleneck, deceptive distance), los obstáculos fuerza desvíos y árboles de búsqueda mucho más grandes.
- **Costo cómputo:** Pasar de 2 a 4 replies genera un crecimiento exponencial en los nodos explorados.
- **Efecto Horizonte y Prudencia:**
 - Acierto: a 4 plies, MAX “ve más lejos”, anticipa los bloqueos de MIN y gana (bottleneck).
 - Sobreprotección: A 4 plies, MAX detecta respuestas de MIN que lo atrapan si avanza; para evitar la gran penalización de perder (-1000), prefiere duda y agotar las 40 rondas. (deceptive distance)

Poda alfa-beta y función de evaluación

El algoritmo de Alfa-Beta garantiza encontrar la acción óptima, y esto lo hace porque mantiene dos cotas dinámicas durante su búsqueda. Por un lado, alfa, el cual es el mejor valor

“más alto” que el jugador MAX (defensor) tiene garantizado hasta el momento, y el contrario, beta, que es el mejor valor “más bajo” que el jugador MIN (intruso) tiene garantizado hasta el momento. Un factor importante a tener en cuenta en este algoritmo, es el criterio para podar, donde una rama se descarta tan pronto cuando alfa sea mayor o igual a beta.

Para lograr esto, se implementa un caso base recursivo que se detiene en tres condiciones: victoria, derrota o al alcanzar la profundidad límite. En cualquiera de estos escenarios, el nodo retorna el valor de *evaluation_function(current_state)*. Tanto el nodo MAX como el nodo MIN intentarán sobre los sucesores intentando maximizar y minimizar la cota respectivamente. Por último, la raíz ejecuta el bucle inicial asignando al Agente MAX, manteniendo las cotas globales iniciales.

En cuanto a la estructura de *evaluation_function*, esta es una función calcula una puntuación numérica para calificar qué tan favorable es un estado del juego y una bonificación por movilidad, mientras aplica penalizaciones proporcionales a la distancia del intruso y su cercanía a las terminales pendientes. Sus características y pesos se listan a continuación:

- **Puntaje actual** ($puntaje * 2.0$): premia el progreso general que se ha alcanzado en la partida; se multiplica por 2.0 para tener mayor relevancia que la movilidad básica, de esta forma, se asegura que las acciones que sumen puntos se prioricen de manera consistente.
- **Proximidad al intruso** ($- 3.0 * distancia\ intruso$): utiliza las distancias reales respetando los muros; el asignar un peso negativo se castiga la lejanía, obligando al defensor a interceptar al intruso.
- **Terminales pendientes** ($- 2.0 * peligro$): evalúa la distancia del intruso a la terminal activa más cercana. Si el intruso se acerca a unos pasos determinados, se da una penalización que desvía la prioridad del defensor hacia la cobertura.
- **Movilidad** ($0.5 * |acciones|$): da un peso positivo pequeño por conservar un mayor número de acciones legales, lo que evita que el agente tome decisiones que lo dejan atrapado.

Adicionalmente, para cumplir con la restricción de que ningún estado no terminal, por más favorable que parezca, alcance +1000.0, ni que un estado desfavorable caiga a -1000.0 al acotar el resultado final con $\max(-999.0, \min(999.0, valor_final))$.

Dual terminal

Se evaluó la equivalencia de decisiones y la eficiencia de la poda comparando MinimaxAgent y AlphaBetaAgent a profundidades 2 y 4.

Profundidad	Algoritmo	Acción Inicial	Puntaje Final
2	Minimax	South	-40.0
	Alfa-Beta	South	-40.0
4	Minimax	East	60.0
	Alfa-Beta	East	60.0

Con profundidad 2 ambos agentes seleccionaron la acción South, mientras que con profundidad 4 ambos cambiaron a East. El puntaje final se mantuvo igual en ambos, confirmando que Alfa-Beta respeta la optimalidad de Minimax. Adicionalmente, como se esperaba, hay un ahorro computacional en Alfa-Beta pues requiere menos nodos a explorar.

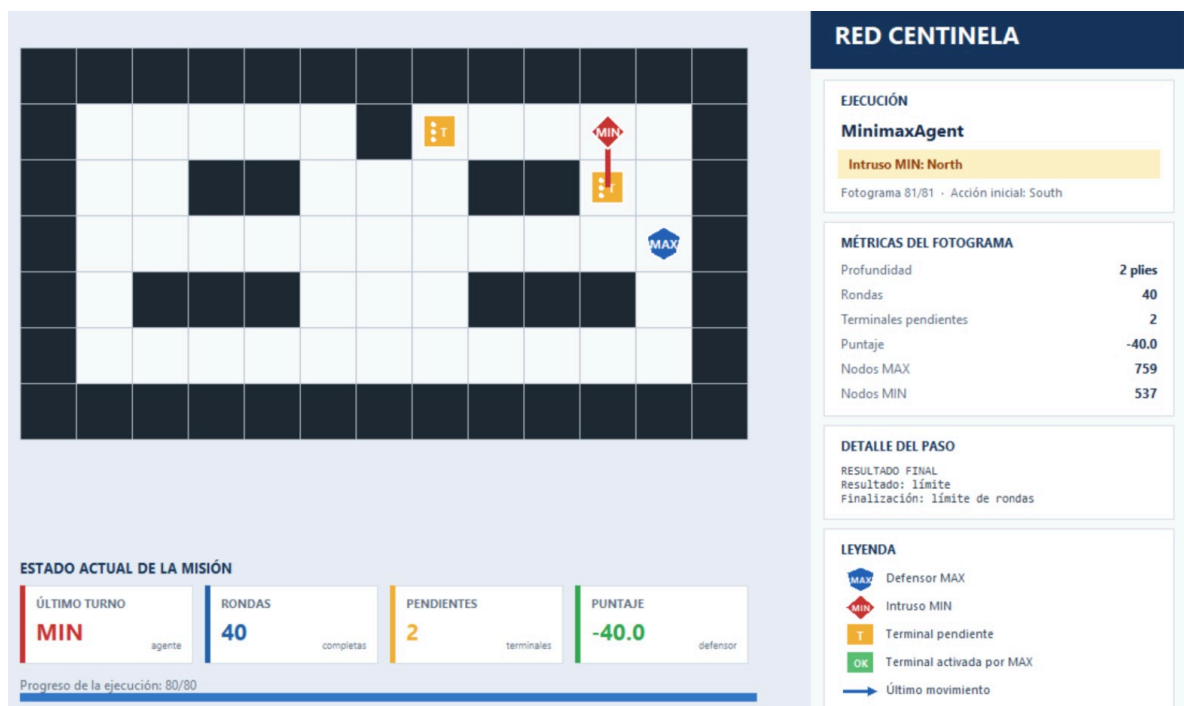


Imagen 34. Dual terminal Minimax profundidad 2

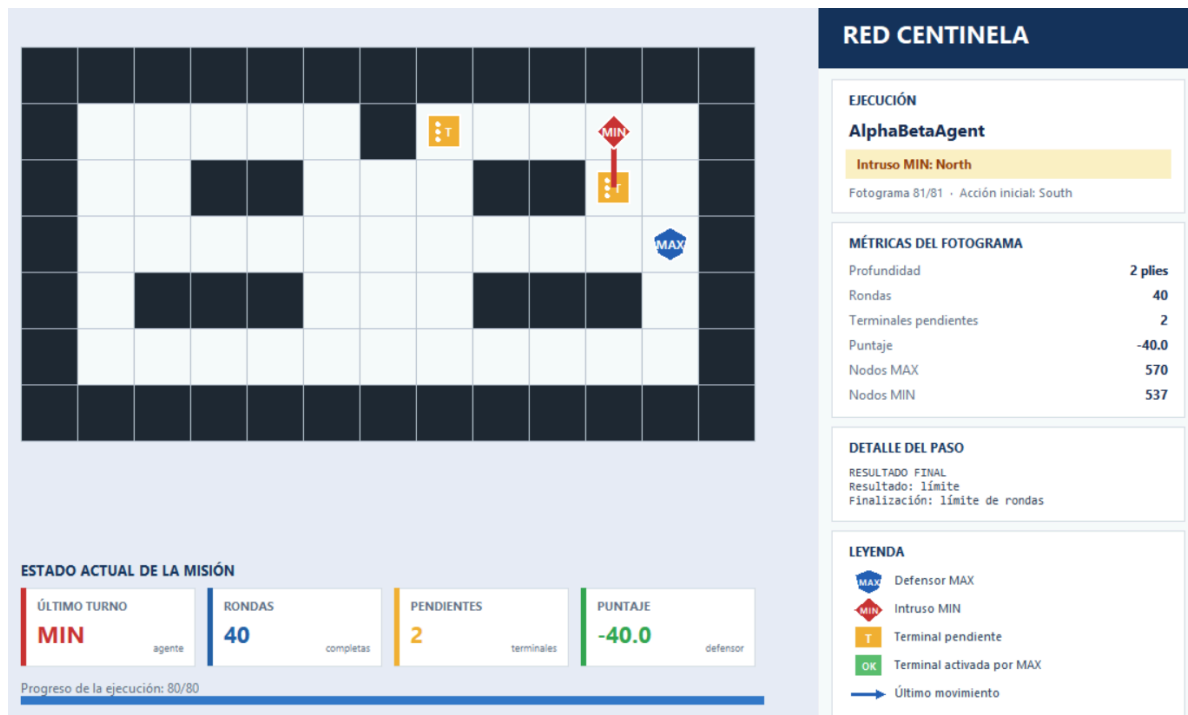


Imagen 35. Dual terminal Alfa-Beta profundidad 2

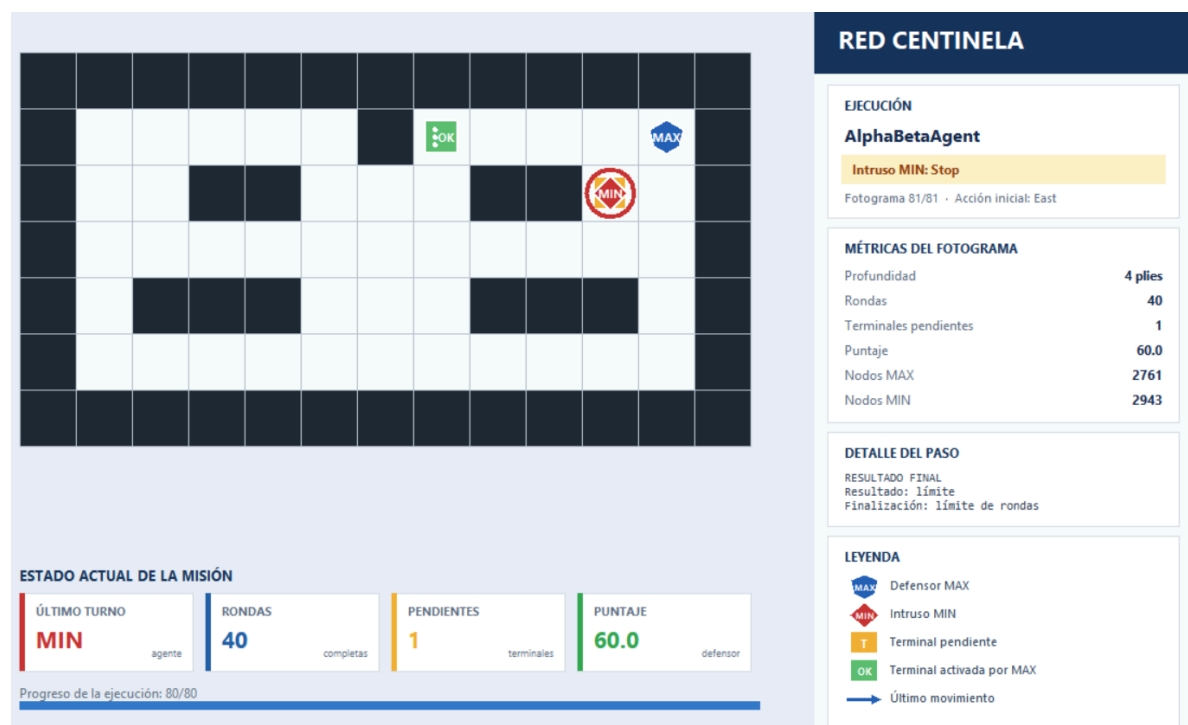


Imagen 36. Dual terminal Alfa-Beta profundidad 4

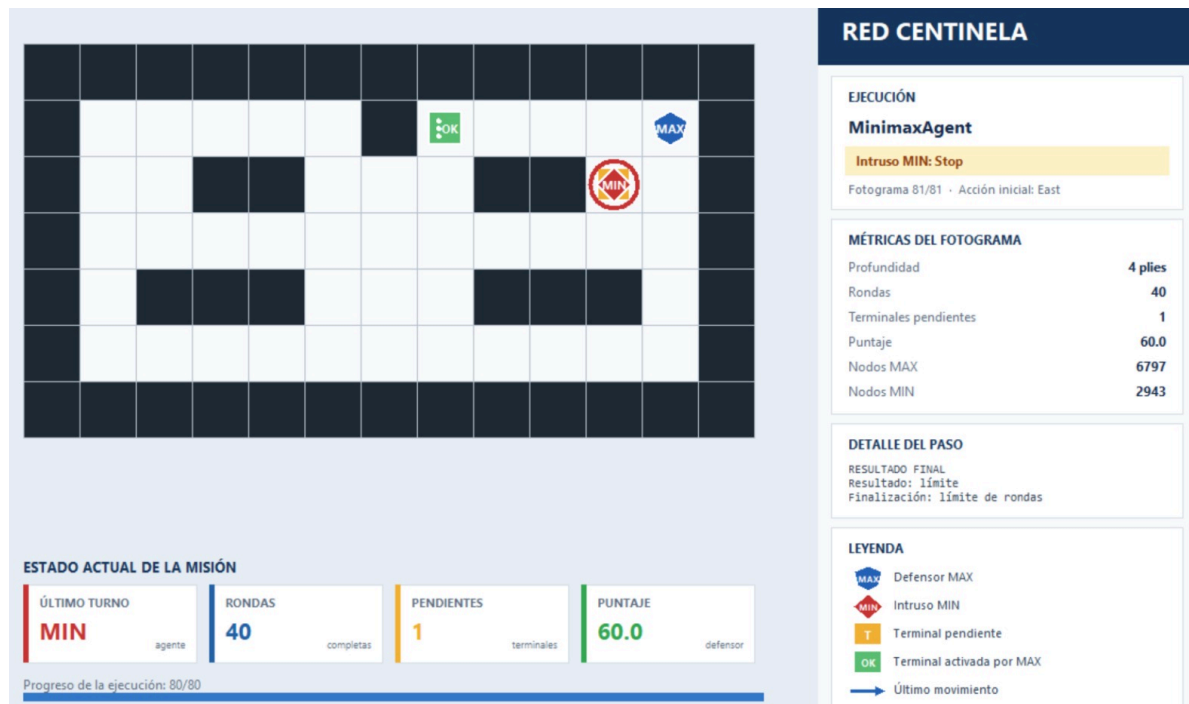


Imagen 37. Dual terminal Minimax profundidad 4

Medium defense

Se evaluó la equivalencia de decisiones y la eficiencia de la poda comparando MinimaxAgent y AlphaBetaAgent a profundidades 2 y 4.

Profundidad	Algoritmo	Acción Inicial	Puntaje Final
2	Minimax	North	260.0
	Alfa-Beta	North	260.0
4	Minimax	North	160.0
	Alfa-Beta	North	160.0

Con ambas profundidades, la acción inicial seleccionada por el defensor fue North, los puntajes finales en ambas profundidades resultaron iguales entre Minimax y Alfa-Beta. En cuanto al impacto de la profundidad en la poda, con medium defense se tiene un mapa con mayor ramificación, elevar la profundidad incrementó el árbol de Minimax a 14,406 nodos. La poda de Alfa-Beta logró descartar subárboles reduciendo la evaluación a 8,917 nodos, reduciendo la carga computación.

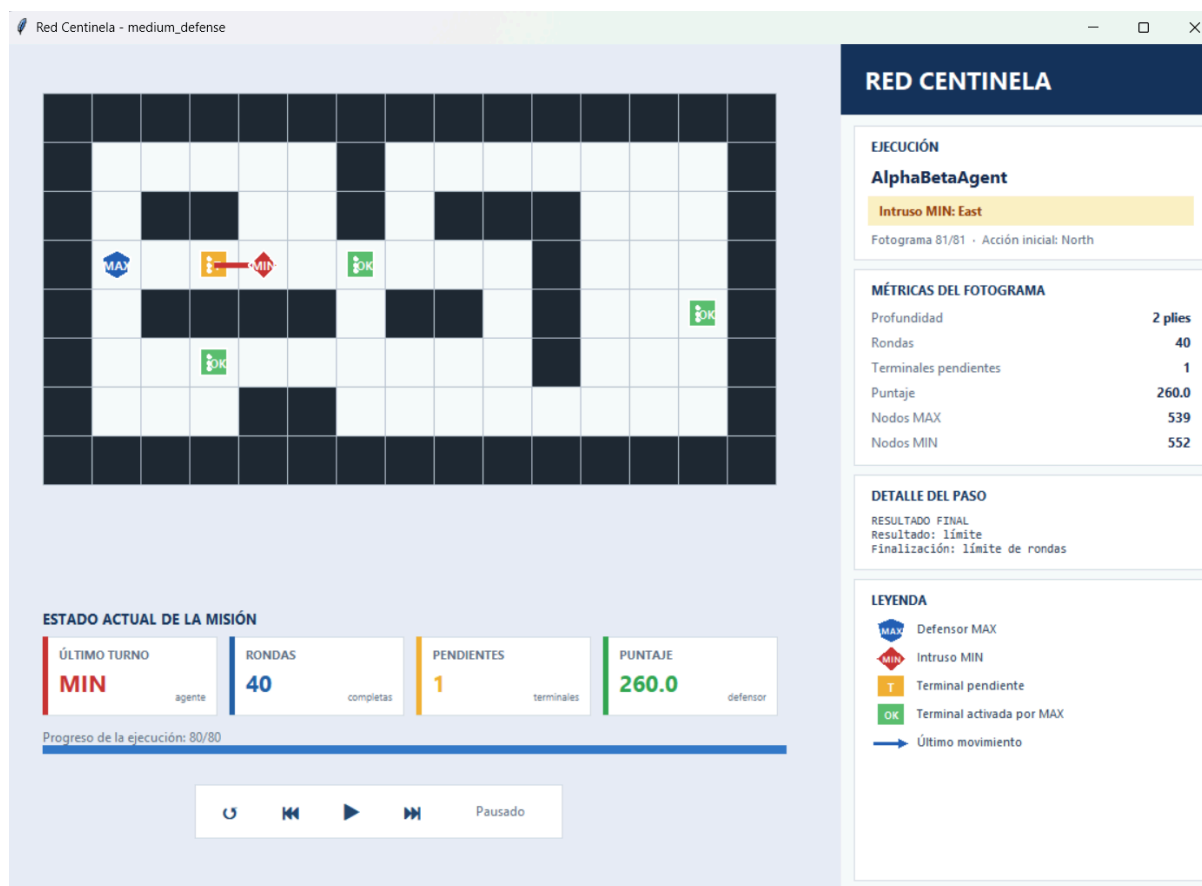


Imagen 38. Medium defense Alfa-Beta profundidad 2

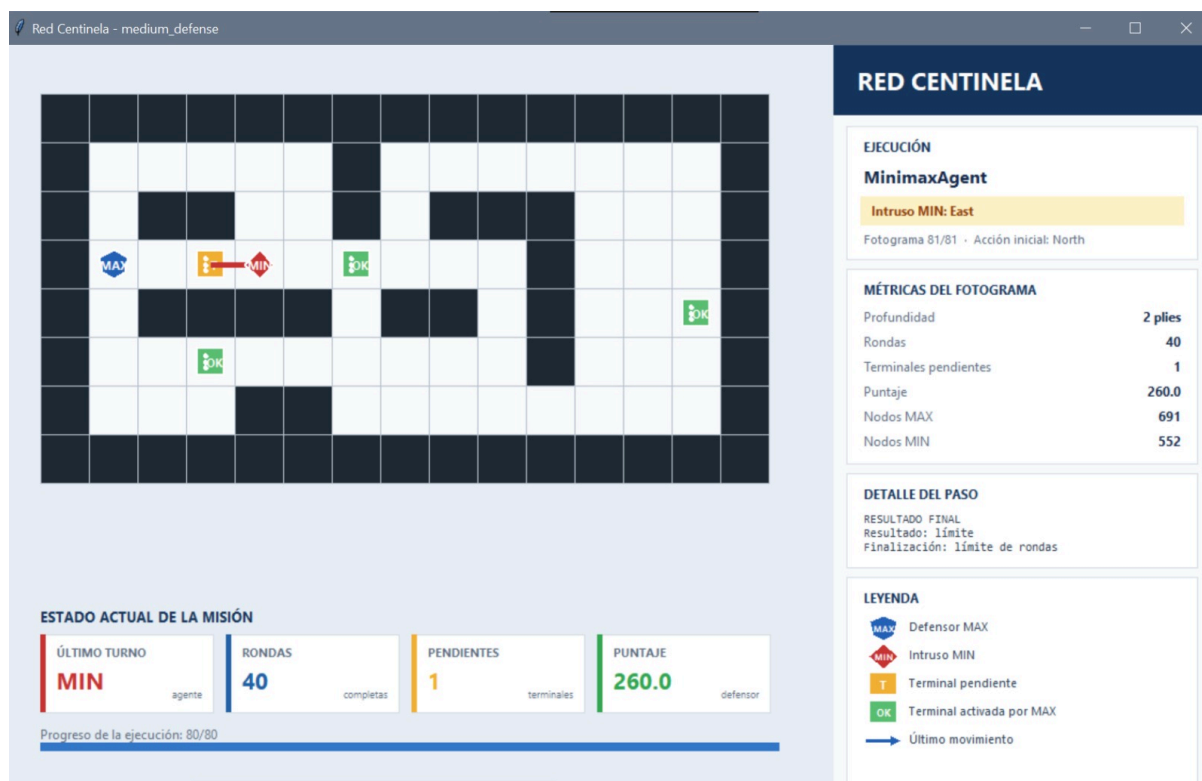


Imagen 39. Medium defense Minimax profundidad 2

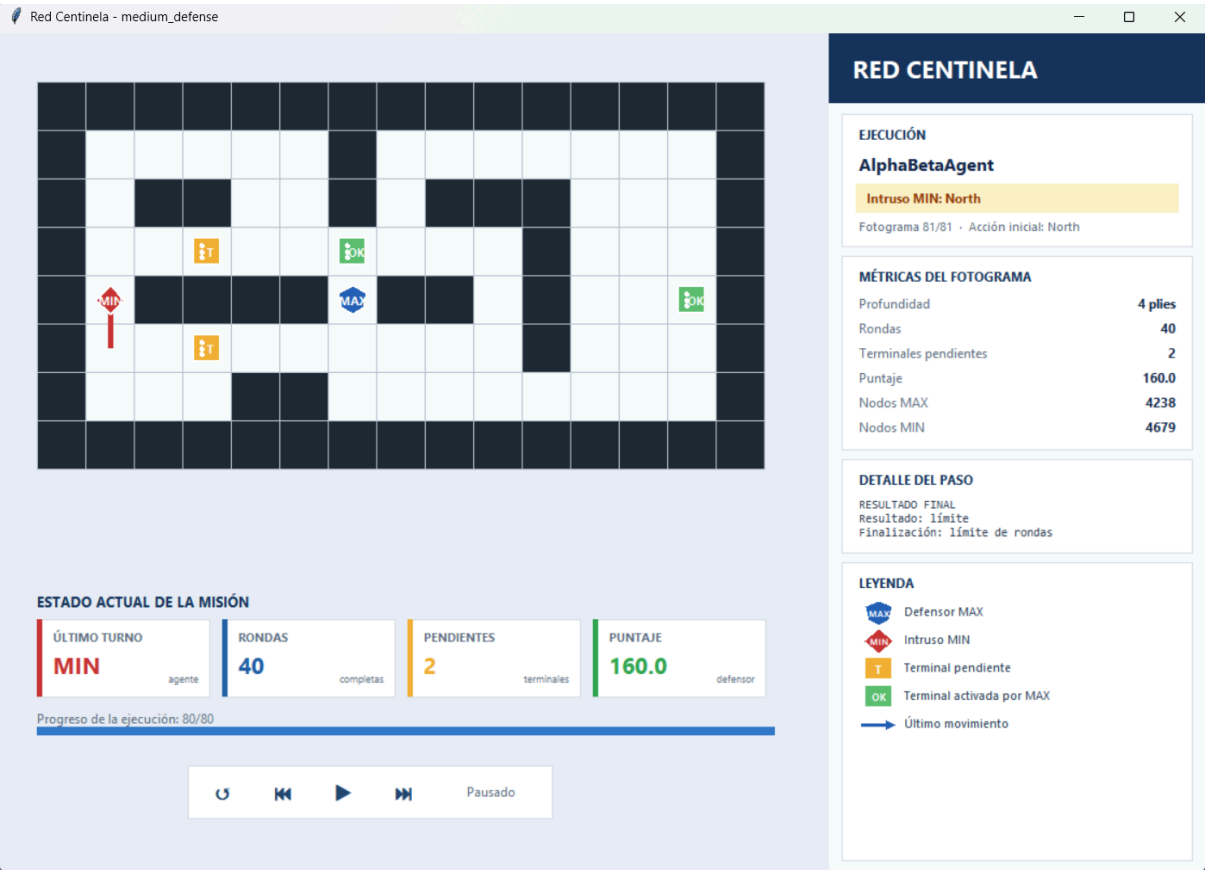


Imagen 40. Medium defense Alfa-Beta profundidad 4

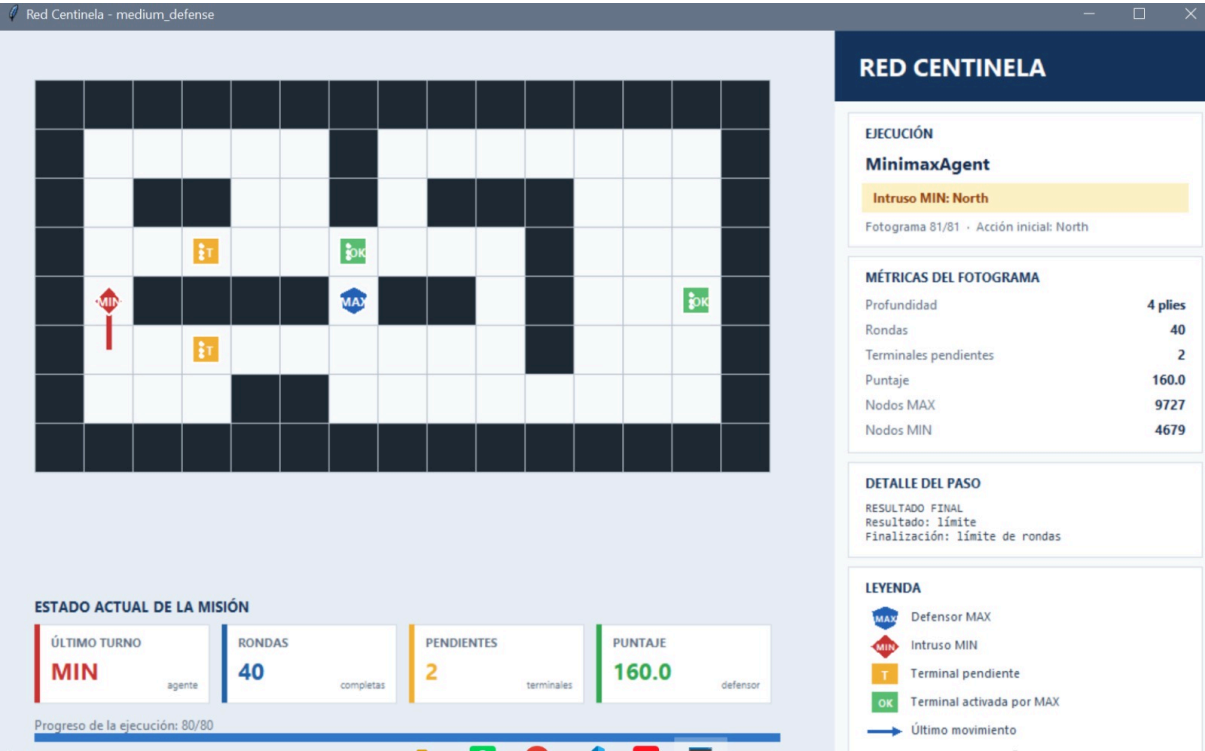


Imagen 41. Medium defense Minimax profundidad 4

Uso IA

Para el punto 5 sobre el diseño de la función de evaluación y la Poda Alfa-Beta se consultó a la IA con respecto al acotamiento del rango $[-1000, 1000]$. El objetivo de esta consulta fue garantizar que las valoraciones de los estados no terminales no iguallen ni superen las utilidades de victoria (1000) o derrota (-1000), evitando que la heurística interfiera con las decisiones óptimas de búsqueda. De este modo, el prompt utilizado fue el siguiente:

“Tengo lista la función `evaluation_function` con todas las distancias y componentes tácticos, pero quiero asegurarme de que el valor final NUNCA iguale ni supere las utilidades terminales (-1000 y +1000).

¿Hace `valor_final = max(-1000.0, min(1000.0, valor_final))` al final del código es suficiente para garantizar esto, o hay algún caso borde donde el rango se pueda romper?”

Respuesta dada por la IA:

No, no es suficiente usar `max(-1000.0, min(1000.0, valor_final))` si tu objetivo explícito es que un estado no terminal nunca iguale a -1000 o 1000. Ese clamping limita el valor al intervalo cerrado $[-1000.0, 1000.0]$, por lo que si los componentes suman un valor como 1050.0, el resultado será exactamente 1000.0, rompiendo el requerimiento de estricta desigualdad $(-1000, 1000)$.

Casos borde donde tu código falla:

1. Colisión directa con las utilidades terminales:
 - a. Si la combinación táctica da mayor o igual a 1000.0 o menor o igual a -1000.0, la función devolverá 1000.0 o -1000.0. El algoritmo Minimax/Alfa-Beta interpretará erróneamente un estado de corte intermedio como si fuera una victoria o derrota absoluta.
2. Floats de tipo NaN:
 - a. Si por algún cálculo previo `valor_final` se convierte en `math.nan`, las comparaciones con `min` y `max` en Python devuelven `NaN`. Retornar `NaN` rompe por completo las comparaciones en Alfa-Beta.
3. Manejo de inf:
 - a. Si `valor_final` llega como `float('inf')`, `min(1000.0, inf)` devuelve 1000.0, cayendo nuevamente en el problema del límite

La solución correcta es acotar el rango al intervalo estrictamente abierto $(-1000, 1000)$.

Junto a lo anterior, la IA mencionó incluir esta parte de código:

```
# 1. Proteger contra valores no numéricos
if math.isnan(valor_final):
    valor_final = 0.0

# 2. Restringir estrictamente al rango (-999.0, 999.0)
#     Evita devolver exactamente -1000.0 o 1000.0
MARGEN = 999.0
return max(-MARGEN, min(MARGEN, valor_final))
```

A esta respuesta se preguntó lo siguiente:

“Este sería un equivalente?: $\text{valor_final} = \max(-999.0, \min(999.0, \text{valor_final}))$ ”

A lo que la IA respondió:

“Sí, es exactamente equivalente. Hacer garantiza matemáticamente que el resultado pertenezca al intervalo $[-999.0, 999.0]$, lo que cumple con la regla de estar estrictamente dentro de $(-1000.0, +1000.0)$. ”